

Yes. **Phase 2 starts now.** We'll do it exactly like a beginner course: one small step at a time, and we won't move to the next step until the current one works.

Your Phase 2 goal is:

- ☐ Install n8n
- ☐ Start n8n
- ☐ Create Webhook
- ☐ Test webhook
- ☐ Receive JSON
- ☐ Learn expressions
- ☐ Add validation
- ☐ Add Switch

The learning guide you uploaded is built around the same core model: **Trigger** → **Process** → **Action**, with Webhook as a trigger, data-shaping nodes such as Edit Fields, and branching with Switch.

Phase 2A — Install n8n

Because current n8n installation guidance is changing around the 3.0 release, I recommend using **Docker** for a new self-hosted installation. n8n's current documentation says new installations should use Docker from 3.0 onward; the older npm/npx approach is being deprecated. ([GitHub]^[1])

Step A1 — Install Docker Desktop

Download Docker Desktop:

[Docker Desktop](#)

Install it normally and start Docker Desktop.

You should eventually see Docker running.

Step A2 — Open PowerShell

On Windows:

```
Start  
→ PowerShell
```

Check Docker:

```
docker --version
```

You should get something similar to:

```
Docker version ...
```

Then check:

```
docker compose version
```

You should also get a version.

Step A3 — Create a folder for n8n

In PowerShell:

```
mkdir C:\n8n  
cd C:\n8n
```

Step A4 — Create `docker-compose.yml`

Run:

```
notepad docker-compose.yml
```

Paste:

```
services:  
  n8n:  
    image: docker.n8n.io/n8nio/n8n  
    ports:  
      - "5678:5678"  
    volumes:  
      - n8n_data:/home/node/.n8n  
    restart: unless-stopped  
  
volumes:  
  n8n_data:
```

Save the file and close Notepad.

Step A5 — Start n8n

In PowerShell:

```
docker compose up -d
```

Then:

```
docker compose ps
```

You should see the n8n container running.

Open:

```
http://localhost:5678
```

n8n's documentation uses `localhost:5678` for a local instance. ([n8n]^[2])

Step A6 — Create your n8n owner account

The first time you open n8n, create your owner account.

Don't use your Meta access tokens here.

Your n8n login is completely separate from your Instagram/Facebook credentials.

Phase 2B — Create your first workflow

Once n8n opens:

Step B1

Click:

Create Workflow

Name it:

```
Social Publisher - Phase 2
```

We are **not** connecting Instagram or Facebook yet.

First we're learning the n8n plumbing.

Phase 2C — Create Webhook

The Webhook is our **Trigger**.

This matches the learning guide's Trigger → Process → Action structure.

Step C1

Click:

Add first step

Search:

Webhook

Select:

Webhook

Step C2 — Configure it

Set:

HTTP Method

POST

Path

social-publisher

So you'll have a URL similar to:

`http://localhost:5678/webhook-test/social-publisher`

n8n creates separate **Test** and **Production** webhook URLs. For development, use the **Test URL**. n8n says the Test URL works while you are listening for a test event, and it stays active for 120 seconds. ([GitHub]^[3])

Phase 2D — Test the webhook

This is your first real n8n API test.

Step D1

In the Webhook node, click:

Listen for test event

You should see n8n waiting for a request.

Step D2

Open Postman.

Create:

POST

to your n8n **Test URL**:

```
http://localhost:5678/webhook-test/social-publisher
```

Your exact URL may be shown inside n8n; **use the one n8n gives you**.

In Postman:

Body → **raw** → **JSON**

Send:

```
{
  "caption": "Hello from my first n8n webhook",
  "image_url": "https://example.com/test.jpg",
  "platform": "instagram"
}
```

Click **Send**.

Step D3

Go back to n8n.

The Webhook node should now show the incoming data.

You should see something roughly like:

```
{
  "body": {
    "caption": "Hello from my first n8n webhook",
    "image_url": "https://example.com/test.jpg",
    "platform": "instagram"
  }
}
```

The exact wrapper can vary depending on the webhook/request configuration.

What you just learned

```
Postman
  ↓
HTTP POST
  ↓
n8n Webhook
  ↓
JSON input
```

That is the foundation of your actual application.

Your frontend will eventually replace Postman:

```
Frontend
  ↓
POST
  ↓
n8n Webhook
```

Phase 2E — Understand the JSON

This is where n8n starts becoming useful.

Suppose the webhook receives:

```
{
  "body": {
    "caption": "My new post",
    "image_url": "https://example.com/photo.jpg",
    "platform": "instagram"
  }
}
```

n8n expressions can access the data using `$json`. The current n8n documentation gives examples such as `{{ $json.body.city }}`. ([GitHub]^[4])

So:

Caption

```
{{ $json.body.caption }}
```

Image URL

```
{{ $json.body.image_url }}
```

Platform

```
{{ $json.body.platform }}
```

This is one of the most important concepts in n8n.

Phase 2F — Add Edit Fields

Your learning guide places data shaping in the **Process** part of the workflow.

Add:

```
Webhook
↓
Edit Fields
```

Search:

Edit Fields

Create these fields

caption

Value:

```
{{ $json.body.caption }}
```

image_url

Value:

```
{{ $json.body.image_url }}
```

platform

Value:

```
{{ $json.body.platform }}
```

You are essentially converting:

```
Webhook raw request
```

into:

```
{
  "caption": "My new post",
  "image_url": "https://example.com/photo.jpg",
  "platform": "instagram"
}
```

That makes the rest of the workflow easier to understand.

n8n's current documentation recommends Edit Fields when your goal is to add/modify/remove fields using expressions. ([GitHub]^[5])

Phase 2G — Learn expressions properly

The basic syntax is:

```
{{ ... }}
```

Examples:

```
{{ $json.body.caption }}
```



```
{{ $json.body.image_url }}
```

```
{{ $json.body.platform }}
```

The current n8n expression reference also provides:

```
$json  
$binary  
$now  
$today  
$("nodeName").first()  
$("nodeName").item
```

among others. ([GitHub]^[6])

For our application, the first three things you need to remember are:

```
$json  
$json.body  
{{ $json.body.caption }}
```

Phase 2H — Add validation

Now we'll make sure bad requests don't continue.

Your workflow becomes:

```
Webhook  
  ↓  
Edit Fields  
  ↓  
IF
```

Add an **IF** node.

We want to check that:

```
caption exists  
AND  
image_url exists
```

```
AND
platform exists
```

For now, let's do them one at a time so you understand what's happening.

First condition

Value 1:

```
{{$.json.caption}}
```

Operation:

```
is not empty
```

Then add the other conditions for:

```
{{$.json.image_url}}
```

and:

```
{{$.json.platform}}
```

Use AND between them.

Conceptually:

```
caption?  _____
image_url? _____
platform? _____
           |
           |— AND → valid
```

Phase 2I — Add Switch

This is the most important step before we touch Meta.

Your application has:

```
Instagram
Facebook
Both
```

So we need branching.

The learning guide specifically covers **Switch** for branching based on values.

Your workflow becomes:

```
Webhook
  ↓
Edit Fields
  ↓
Validation
  ↓
Switch
  ├── instagram
  ├── facebook
  └── both
```

Configure Switch

Add:

Switch

Set the value to:

```
{{ $json.platform }}
```

Then configure the three cases:

```
instagram
facebook
both
```

So:

```
platform = instagram
  ↓
Instagram branch

platform = facebook
  ↓
Facebook branch

platform = both
  ↓
Both branches
```

Phase 2J — Test the Switch

Use Postman again.

Test 1

Send:

```
{
  "caption": "Instagram test",
  "image_url": "https://example.com/photo.jpg",
  "platform": "instagram"
}
```

It should go to:

Instagram

Test 2

Send:

```
{
  "caption": "Facebook test",
  "image_url": "https://example.com/photo.jpg",
  "platform": "facebook"
}
```

It should go to:

Facebook

Test 3

Send:

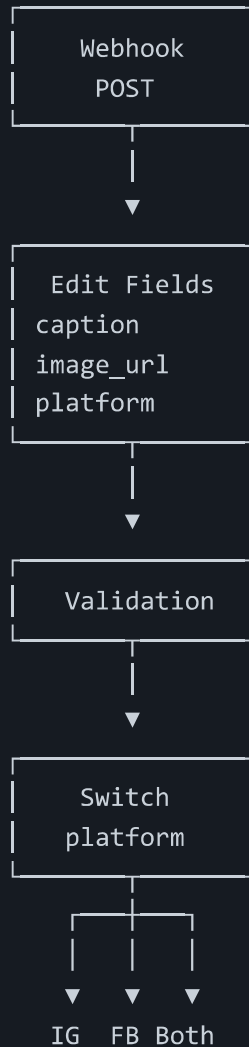
```
{
  "caption": "Both platforms",
  "image_url": "https://example.com/photo.jpg",
  "platform": "both"
}
```

It should go to:

Both

Phase 2 final workflow

When Phase 2 is finished, your n8n workflow should look approximately like:



That is the exact foundation we'll use in Phase 3.

Your Phase 2 checklist

- ☐ Install Docker
- ☐ Start Docker
- ☐ Start n8n
- ☐ Open <http://localhost:5678>
- ☐ Create owner account

- ☐ Create workflow
- ☐ Add Webhook
- ☐ Set POST
- ☐ Set path: social-publisher
- ☐ Listen for test event
- ☐ Send Postman JSON
- ☐ Receive JSON in n8n
- ☐ Add Edit Fields
- ☐ Learn {{\$json...}}
- ☐ Add validation
- ☐ Add Switch
- ☐ Test instagram
- ☐ Test facebook
- ☐ Test both

For now, do only the installation. Run `docker --version` and `docker compose version` in PowerShell. Once Docker is ready, send me the output or a screenshot, and we'll do the n8n startup step next.